

Examen

- El primer examen será el **martes 22 de Septiembre**.
- En

Actividades de aprendizaje autónomo -> Guías para exámenes -> Guía para el parcial #1

encontrarán (antes de la siguiente clase) una guía para el examen.

Contenido: Capítulos 1 y 2.

Matrices de covarianza de los blobs

Inicialmente, `make_blobs` genera cada grupo con $\Sigma = I$.

Para los dos primeros blobs se aplica la transformación

$$\mathbf{x}' = \mathbf{x}\mathbf{A}, \quad \mathbf{A} = \begin{bmatrix} 0,374 & 0,95 \\ 0,732 & 0,598 \end{bmatrix}.$$

Por lo tanto,

$$\Sigma_1 = \Sigma_2 = \mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 0,676 & 0,793 \\ 0,793 & 1,260 \end{bmatrix}.$$

El tercer blob únicamente se traslada, por lo que su covarianza no cambia:

$$\Sigma_3 = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Ajuste con Scikit-Learn

```
from sklearn.mixture import GaussianMixture

gm = GaussianMixture(n_components=3, n_init=10, random_state=42)
gm.fit(X)
```

Después de ajustar el modelo podemos inspeccionar:

- `gm.weights_`: pesos $\phi^{(j)}$,
- `gm.means_`: medias $\mu^{(j)}$,
- `gm.covariances_`: matrices de covarianza $\Sigma^{(j)}$.

Nota

En `GaussianMixture` se debe indicar por adelantado el número de componentes k .

Algoritmo EM: Expectation–Maximization

El modelo se ajusta usando el algoritmo EM, que alterna dos pasos hasta converger.

E-step

Estima, para cada instancia, la probabilidad de pertenecer a cada componente.

M-step

Actualiza pesos, medias y covarianzas usando esas probabilidades como pesos.

$$\gamma_{ij} = P(z^{(i)} = j \mid \mathbf{x}^{(i)})$$

Estas probabilidades se llaman **responsabilidades**.

¿Suenan parecido?

EM para Gaussian Mixtures: actualizar clusters

M-step: actualizar los parámetros usando las responsabilidades.

Primero calculamos el “tamaño efectivo” del cluster:

$$N_j = \sum_{i=1}^m \gamma_{ij}$$

EM para Gaussian Mixtures: actualizar clusters

M-step: actualizar los parámetros usando las responsabilidades.

Primero calculamos el “tamaño efectivo” del cluster:

$$N_j = \sum_{i=1}^m \gamma_{ij}$$

Actualizamos el peso del cluster:

$$\phi_j = \frac{N_j}{m}$$

EM para Gaussian Mixtures: actualizar clusters

M-step: actualizar los parámetros usando las responsabilidades.

Primero calculamos el “tamaño efectivo” del cluster:

$$N_j = \sum_{i=1}^m \gamma_{ij}$$

Actualizamos el peso del cluster:

$$\phi_j = \frac{N_j}{m}$$

Actualizamos la media:

$$\mu_j = \frac{1}{N_j} \sum_{i=1}^m \gamma_{ij} x_i$$

EM como generalización de k -means

	k -means	GMM con EM
Asignación	dura	suave
Centro	centroides	medias $\mu^{(j)}$
Forma	implícitamente circular	elipsoidal
Tamaño/densidad	similar	puede variar
Parámetros	centros	pesos, medias, covarianzas

Cuidado

Como k -means, EM puede converger a soluciones pobres. Por eso conviene usar varios reinicios, por ejemplo `n_init=10`.

Clustering duro y suave

Una vez ajustado, el GMM puede asignar clusters de dos formas.

Hard clustering

Asigna cada punto al componente más probable.

```
gm.predict(X)
```

Soft clustering

Devuelve la probabilidad de pertenecer a cada componente.

```
gm.predict_proba(X)
```

La asignación suave es útil cuando los clusters se traslapan.

Modelo generativo y densidad

Un GMM no solo agrupa: también describe una distribución de probabilidad.

- Puede generar nuevas instancias:

```
X_new, y_new = gm.sample(6)
```

- Puede estimar la log-densidad (mayor = más denso) en una ubicación:

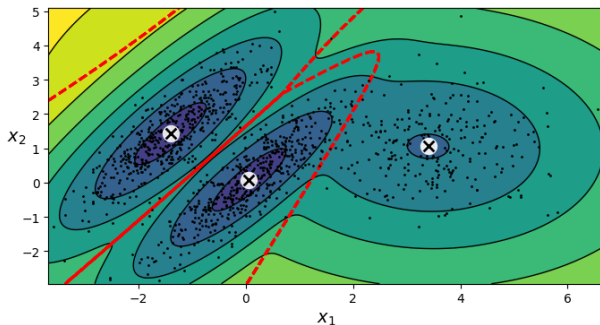
```
gm.score_samples(X)
```

Importante

`score_samples()` devuelve el logaritmo de la función de densidad. Su exponencial es una densidad, no una probabilidad directa.

Densidad, fronteras y contornos

- Las medias indican el centro de cada componente.
- Las fronteras de decisión separan regiones donde domina cada componente.
- Los contornos de densidad muestran zonas más o menos probables.



En datos reales, el reto crece si hay muchas dimensiones, muchos clusters o pocas observaciones.

Restricciones en las covarianzas

Para simplificar el modelo se puede restringir la forma de las matrices de covarianza con `covariance_type`.

<code>covariance_type</code>	Restricción geométrica
<code>spherical</code>	clusters esféricos, con varianzas distintas
<code>diag</code>	elipses alineadas con los ejes
<code>tied</code>	todos comparten la misma covarianza
<code>full (default)</code>	cada cluster tiene covarianza libre

Trade-off

Más flexibilidad puede mejorar el ajuste, pero aumenta el número de parámetros y el costo computacional.

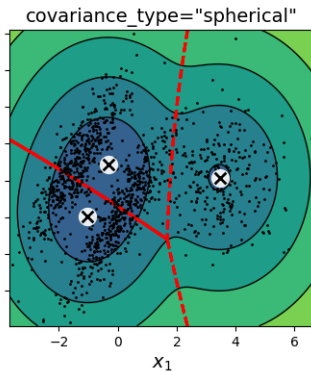
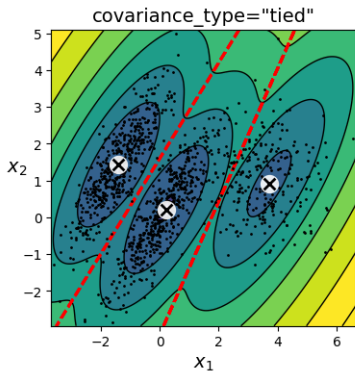
Tipos de covarianza

```
gm_full = GaussianMixture(n_components=3, n_init=10, covariance_type="full",
                           random_state=42)
gm_tied = GaussianMixture(n_components=3, n_init=10, covariance_type="tied",
                           random_state=42)
gm_spherical = GaussianMixture(n_components=3, n_init=10, covariance_type="spherical",
                                random_state=42)
gm_diag = GaussianMixture(n_components=3, n_init=10, covariance_type="diag",
                           random_state=42)

gm_full.fit(X)
gm_tied.fit(X)
gm_spherical.fit(X)
gm_diag.fit(X)

plot_gaussian_mixture(gm_full, X)
plot_gaussian_mixture(gm_tied, X)
plot_gaussian_mixture(gm_spherical, X)
plot_gaussian_mixture(gm_diag, X)
```

Gráficas



Anomalía vs. novedad

Detección de anomalías

El conjunto de entrenamiento puede contener algunos outliers. El método puede usarse para identificarlos y limpiar los datos.

Detección de novedades

El modelo se entrena con datos “limpios”. Luego se buscan observaciones nuevas que no parezcan venir de la distribución aprendida.

Resumen: un GMM aprende pesos, medias y covarianzas; permite clustering suave, estimación de densidad, generación de datos y detección de anomalías.

Idea general: balance ajuste vs. complejidad

Objetivo

Elegir el modelo que explica bien los datos, pero sin usar demasiados parámetros.

Idea general: balance ajuste vs. complejidad

Objetivo

Elegir el modelo que explica bien los datos, pero sin usar demasiados parámetros.

- Más clusters $\Rightarrow$ más parámetros.
- Más parámetros $\Rightarrow$ mayor riesgo de sobreajuste.
- BIC y AIC penalizan la complejidad y recompensan el buen ajuste.

Regla práctica

Se elige el modelo con **menor** BIC o AIC.

Criterios BIC y AIC

Definiciones

$$BIC = \log(m) p - 2 \log(\mathcal{L})$$

$$AIC = 2p - 2 \log(\mathcal{L})$$

Crterios BIC y AIC

Definiciones

$$BIC = \log(m) p - 2 \log(\mathcal{L})$$

$$AIC = 2p - 2 \log(\mathcal{L})$$

- m : número de instancias.
- p : número de parámetros aprendidos por el modelo.
- $\mathcal{L}$: valor máximo de la función de verosimilitud.

Comparación entre BIC y AIC

BIC

- Penaliza la complejidad con $\log(m)p$.
- Suele preferir modelos más simples.
- La penalización crece con el tamaño del dataset.

AIC

- Penaliza la complejidad con $2p$.
- Puede elegir modelos más flexibles.
- Suele ajustar mejor, pero con mayor complejidad.

Comparación entre BIC y AIC

BIC

- Penaliza la complejidad con $\log(m)p$.
- Suele preferir modelos más simples.
- La penalización crece con el tamaño del dataset.

AIC

- Penaliza la complejidad con $2p$.
- Puede elegir modelos más flexibles.
- Suele ajustar mejor, pero con mayor complejidad.

Cuando difieren, BIC tiende a seleccionar un modelo más simple que AIC.

Probabilidad vs. verosimilitud

Probabilidad

Con parámetros fijos θ , mide qué tan plausible es un resultado futuro x :

$$f(x; \theta)$$

Probabilidad vs. verosimilitud

Probabilidad

Con parámetros fijos θ , mide qué tan plausible es un resultado futuro x :

$$f(x; \theta)$$

Verosimilitud

Con datos observados fijos, mide qué tan plausibles son los parámetros:

$$\mathcal{L}(\theta | x) = f(x; \theta)$$

Probabilidad vs. verosimilitud

Probabilidad

Con parámetros fijos θ , mide qué tan plausible es un resultado futuro x :

$$f(x; \theta)$$

Verosimilitud

Con datos observados fijos, mide qué tan plausibles son los parámetros:

$$\mathcal{L}(\theta \mid x) = f(x; \theta)$$

Diferencia clave

La PDF es función de x ; la verosimilitud es función de θ .

Estimación por máxima verosimilitud (MLE)

- Dado un conjunto de datos X , buscamos los parámetros más plausibles.

Estimación por máxima verosimilitud (MLE)

- Dado un conjunto de datos X , buscamos los parámetros más plausibles.
- Esto se hace maximizando la verosimilitud:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta | X).$$

Estimación por máxima verosimilitud (MLE)

- Dado un conjunto de datos X , buscamos los parámetros más plausibles.
- Esto se hace maximizando la verosimilitud:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta | X).$$

- Si incorporamos una distribución previa $g(\theta)$, maximizamos:

$$\mathcal{L}(\theta \mid X)g(\theta).$$

Estimación por máxima verosimilitud (MLE)

- Dado un conjunto de datos X , buscamos los parámetros más plausibles.
- Esto se hace maximizando la verosimilitud:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta | X).$$

- Si incorporamos una distribución previa $g(\theta)$, maximizamos:

$$\mathcal{L}(\theta \mid X)g(\theta).$$

- Esto se llama **MAP**: máxima estimación a posteriori.

MLE vs. MAP

MLE

$$\arg \max_{\theta} \mathcal{L}(\theta \mid X)$$

- Usa solo los datos observados.
- Busca el mejor ajuste posible.

MAP

$$\arg \max_{\theta} \mathcal{L}(\theta | X)g(\theta)$$

- Usa datos y conocimiento previo.
- Puede verse como una versión regularizada de MLE.

Por qué usar log-verosimilitud

- Maximizar $\mathcal{L}$ es equivalente a maximizar $\log \mathcal{L}$, porque el logaritmo es estrictamente creciente.

Por qué usar log-verosimilitud

- Maximizar $\mathcal{L}$ es equivalente a maximizar $\log \mathcal{L}$, porque el logaritmo es estrictamente creciente.
- Con observaciones independientes:

$$\mathcal{L}(\theta | X) = \prod_{i=1}^m \mathcal{L}(\theta | x^{(i)}).$$

Por qué usar log-verosimilitud

- Maximizar $\mathcal{L}$ es equivalente a maximizar $\log \mathcal{L}$, porque el logaritmo es estrictamente creciente.
- Con observaciones independientes:

$$\mathcal{L}(\theta | X) = \prod_{i=1}^m \mathcal{L}(\theta | x^{(i)}).$$

- Al tomar logaritmos:

$$\log \mathcal{L}(\theta | X) = \sum_{i=1}^m \log \mathcal{L}(\theta | x^{(i)}).$$

- Esto es más estable y más fácil de optimizar.

Verosimilitud en un Gaussian Mixture Model

Para una observación $\mathbf{x}_i$, la densidad bajo un GMM con k componentes es:

$$p(\mathbf{x}_i; \theta) = \sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j)$$

Verosimilitud en un Gaussian Mixture Model

Para una observación $\mathbf{x}_i$, la densidad bajo un GMM con k componentes es:

$$p(\mathbf{x}_i; \theta) = \sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j)$$

Si las m observaciones son independientes, la función de verosimilitud es:

$$\mathcal{L}(\theta; X) = \prod_{i=1}^m \left[\sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \right]$$

Verosimilitud en un Gaussian Mixture Model

Para una observación $\mathbf{x}_i$, la densidad bajo un GMM con k componentes es:

$$p(\mathbf{x}_i; \theta) = \sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j)$$

Si las m observaciones son independientes, la función de verosimilitud es:

$$\mathcal{L}(\theta; X) = \prod_{i=1}^m \left[\sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \right]$$

En la práctica se maximiza la **log-verosimilitud**:

$$\log \mathcal{L}(\theta; X) = \sum_{i=1}^m \log \left[\sum_{j=1}^k \phi_j \mathcal{N}(\mathbf{x}_i | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \right]$$

Idea

Para cada punto se suman las contribuciones de todas las gaussianas; EM busca los parámetros θ que maximizan esta verosimilitud.

Calcular BIC y AIC en Scikit-Learn

Después de ajustar una mezcla gaussiana:

```
from sklearn.mixture import GaussianMixture

gm = GaussianMixture(n_components=3, n_init=10,
                      random_state=42)

gm.fit(X)

bic = gm.bic(X)
aic = gm.aic(X)
```

Calcular BIC y AIC en Scikit-Learn

Después de ajustar una mezcla gaussiana:

```
from sklearn.mixture import GaussianMixture

gm = GaussianMixture(n_components=3, n_init=10,
                     random_state=42)
gm.fit(X)

bic = gm.bic(X)
aic = gm.aic(X)
```

- `bic(X)` calcula el criterio BIC.
- `aic(X)` calcula el criterio AIC.
- Se repite para distintos valores de k y se elige el mínimo.

Procedimiento práctico para elegir k

1. Elegir un rango de valores candidatos: $k = 1, 2, \dots, K$.

Procedimiento práctico para elegir k

1. Elegir un rango de valores candidatos: $k = 1, 2, \dots, K$.
2. Ajustar un modelo GaussianMixture para cada k .

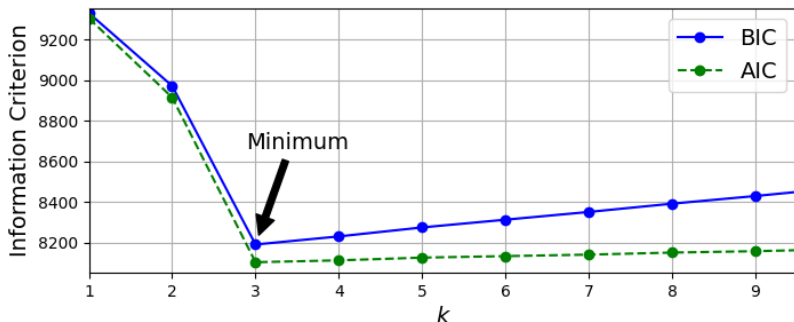
Procedimiento práctico para elegir k

1. Elegir un rango de valores candidatos: $k = 1, 2, \dots, K$.
2. Ajustar un modelo GaussianMixture para cada k .
3. Calcular BIC y/o AIC para cada modelo.

Procedimiento práctico para elegir k

1. Elegir un rango de valores candidatos: $k = 1, 2, \dots, K$.
2. Ajustar un modelo GaussianMixture para cada k .
3. Calcular BIC y/o AIC para cada modelo.
4. Graficar los criterios contra k .

Ejemplo



Si ambos criterios son mínimos en $k = 3$, entonces $k = 3$ es probablemente una buena elección.

Complejidad computacional: clustering

Sea:

- m : número de instancias,
- n : número de dimensiones,
- k : número de componentes,
- l : número de iteraciones.

Algoritmo	Complejidad	Comentario
K-means	$O(l \times kmn)$	Muy rápido en la práctica
DBSCAN	$O(m^2n)$	No escala bien a datasets grandes
GMM: spherical, diag	$O(l \times kmn)$	Covarianzas simples
GMM: tied, full	$O(l \times (kmn^2 + kn^3))$	Covarianzas más flexibles

Idea: mientras más flexible sea la forma del cluster, más parámetros hay que estimar.

K-means $\approx$ centros GMM full $\approx$ centros + formas + orientaciones + pesos

Clustering jerárquico¹

Una posible desventaja de K -means es que debemos especificar de antemano el número de clusters K .

El **clustering jerárquico** es una alternativa que:

- No requiere elegir inicialmente un número de clusters.
- Produce una representación gráfica en forma de árbol llamada **dendrograma**.

¹ An Introduction to Statistical Learning: with Applications in Python - Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor

El dendrograma

Un **dendrograma** representa las observaciones y las fusiones sucesivas realizadas por el algoritmo.

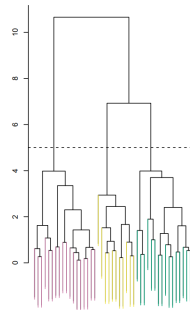
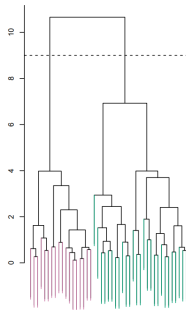
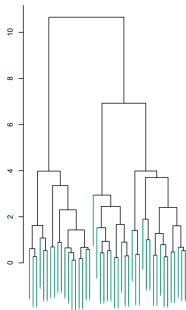
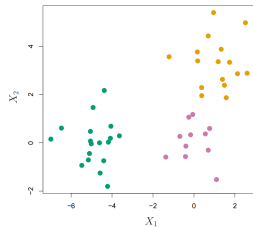
- Cada **hoja** representa una observación.

El dendrograma

Un **dendrograma** representa las observaciones y las fusiones sucesivas realizadas por el algoritmo.

- Cada **hoja** representa una observación.
- Al subir en el árbol, las observaciones y clusters se van fusionando.

Datos y dendograma



Interpretación de un dendrograma

Importante

La proximidad horizontal entre dos observaciones en un dendrograma **no indica necesariamente que sean similares.**

Interpretación de un dendrograma

Importante

La proximidad horizontal entre dos observaciones en un dendrograma **no indica necesariamente que sean similares**.

La similitud se determina observando la **altura de la primera fusión** de las ramas que contienen a las observaciones.

Interpretación de un dendrograma

Importante

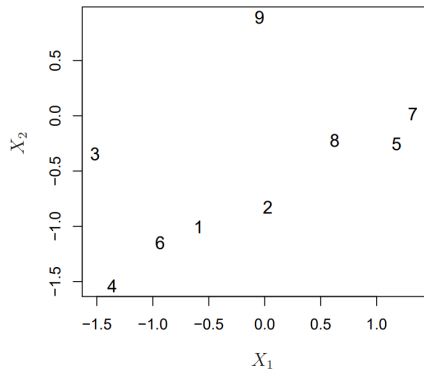
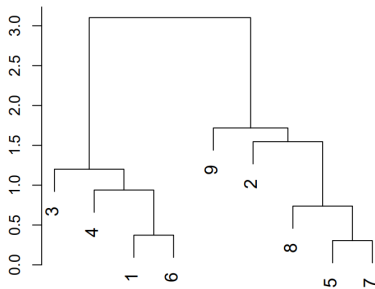
La proximidad horizontal entre dos observaciones en un dendrograma **no indica necesariamente que sean similares.**

La similitud se determina observando la **altura de la primera fusión** de las ramas que contienen a las observaciones.

Por ejemplo:

- Si dos observaciones se fusionan muy abajo, son muy similares.
- Si únicamente se unen cerca de la parte superior del dendrograma, son relativamente diferentes.

Ejemplo



Obtener clusters a partir del dendrograma

Para obtener clusters se realiza un **corte horizontal** en el dendrograma.

Una ventaja del clustering jerárquico

Un mismo dendrograma permite obtener diferentes números de clusters.

Una ventaja del clustering jerárquico

Un mismo dendrograma permite obtener diferentes números de clusters.

Por ejemplo, podemos cortar el mismo árbol para obtener:

$$K = 2, \quad K = 3, \quad K = 4, \quad \dots$$

Estructura jerárquica

El término **jerárquico** significa que los clusters obtenidos mediante un corte deben estar contenidos dentro de los clusters obtenidos mediante un corte a mayor altura.

Estructura jerárquica

El término **jerárquico** significa que los clusters obtenidos mediante un corte deben estar contenidos dentro de los clusters obtenidos mediante un corte a mayor altura.

Es decir, las particiones están **anidadas**.

cluster grande $\supset$ clusters más pequeños $\supset$ observaciones.

Esta suposición puede no ser apropiada para todos los conjuntos de datos.

Una limitación de la estructura jerárquica

Supongamos que tenemos personas:

- hombres y mujeres;
- estadounidenses, japoneses y franceses.

Una limitación de la estructura jerárquica

Supongamos que tenemos personas:

- hombres y mujeres;
- estadounidenses, japoneses y franceses.

La mejor partición en dos grupos podría ser:

{hombres}, {mujeres},

mientras que la mejor partición en tres grupos podría ser:

{estadounidenses}, {japoneses}, {franceses}.

Algoritmo de clustering jerárquico

Supongamos que tenemos n observaciones.

1. Calcular las

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

disimilitudes entre pares de observaciones.

Algoritmo de clustering jerárquico

Supongamos que tenemos n observaciones.

1. Calcular las

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

disimilitudes entre pares de observaciones.

2. Inicialmente, considerar cada observación como un cluster:

$$C_1 = \{x_1\}, \dots, C_n = \{x_n\}.$$

Algoritmo de clustering jerárquico

Supongamos que tenemos n observaciones.

1. Calcular las

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

disimilitudes entre pares de observaciones.

2. Inicialmente, considerar cada observación como un cluster:

$$C_1 = \{x_1\}, \dots, C_n = \{x_n\}.$$

3. Encontrar los dos clusters más similares y fusionarlos.

Algoritmo de clustering jerárquico

Supongamos que tenemos n observaciones.

1. Calcular las

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

disimilitudes entre pares de observaciones.

2. Inicialmente, considerar cada observación como un cluster:

$$C_1 = \{x_1\}, \dots, C_n = \{x_n\}.$$

3. Encontrar los dos clusters más similares y fusionarlos.
4. Actualizar las disimilitudes entre los clusters restantes.
5. Repetir hasta que todas las observaciones pertenezcan a un único cluster.

¿Cómo medimos la distancia entre clusters?

Para observaciones individuales podemos utilizar, por ejemplo, distancia Euclidiana:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}.$$

Pero después de fusionar observaciones surge una nueva pregunta:

¿Cómo definimos la disimilitud entre **dos clusters**?

Complete linkage

El **complete linkage** utiliza la mayor disimilitud entre observaciones pertenecientes a clusters diferentes:

$$d_{complete}(A, B) = \max_{\substack{i \in A \\ j \in B}} d(x_i, x_j)$$

Complete linkage

El **complete linkage** utiliza la mayor disimilitud entre observaciones pertenecientes a clusters diferentes:

$$d_{complete}(A, B) = \max_{\substack{i \in A \\ j \in B}} d(x_i, x_j)$$

Es decir:

1. Se calculan todas las distancias entre puntos de A y puntos de B .
2. Se toma la **mayor** de esas distancias.

Single linkage

El **single linkage** utiliza la menor disimilitud entre observaciones pertenecientes a clusters diferentes:

$$d_{single}(A, B) = \min_{\substack{i \in A \\ j \in B}} d(x_i, x_j)$$

Average linkage

El **average linkage** utiliza el promedio de todas las disimilitudes entre observaciones de los dos clusters:

$$d_{average}(A, B) = \frac{1}{|A||B|} \sum_{i \in A} \sum_{j \in B} d(x_i, x_j)$$

Por lo tanto, considera simultáneamente todas las relaciones entre los puntos de ambos clusters.

Centroid linkage

Sea

$$\bar{x}_A = \frac{1}{|A|} \sum_{i \in A} x_i$$

el centroide del cluster A , y de manera análoga $\bar{x}_B$ para B .

Comparación de criterios de linkage

Linkage	Disimilitud entre A y B
Complete	Mayor distancia entre dos observaciones de clusters diferentes.
Single	Menor distancia entre dos observaciones de clusters diferentes.
Average	Promedio de todas las distancias entre observaciones de clusters diferentes.
Centroid	Distancia entre los centroides de ambos clusters.

Comparación de criterios de linkage

Linkage	Disimilitud entre A y B
Complete	Mayor distancia entre dos observaciones de clusters diferentes.
Single	Menor distancia entre dos observaciones de clusters diferentes.
Average	Promedio de todas las distancias entre observaciones de clusters diferentes.
Centroid	Distancia entre los centroides de ambos clusters.

Average y complete linkage suelen generar clusters más **balanceados** que single linkage.

El linkage importa

El dendrograma obtenido puede cambiar considerablemente dependiendo del criterio de linkage.

Ejemplo: historial de compras

Supongamos una matriz donde

- cada fila representa un cliente;
- cada columna representa un producto;
- x_{ij} es el número de veces que el cliente i compró el producto j .

¿Debemos estandarizar las variables?

La escala de las variables puede tener un efecto importante sobre las disimilitudes.

¿Siempre debemos estandarizar?

No necesariamente.

Clustering jerárquico: decisiones principales

Para aplicar clustering jerárquico debemos decidir:

1. ¿Cómo representar o escalar las variables?
2. ¿Qué medida de disimilitud utilizar?

Para implementar clustering jerárquico utilizamos `AgglomerativeClustering` de `sklearn.cluster`.

```
from sklearn.cluster import AgglomerativeClustering
HClust = AgglomerativeClustering
```

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(0);
X = np.random.standard_normal((50,2));
X[:25,0] += 3;
X[:25,1] -= 4;

plt.scatter(X[:,0],X[:,1]);
```

135 / 160

Complete linkage

Primero realizamos clustering jerárquico utilizando **complete linkage**:

```
hc_comp = HClust(
    distance_threshold=0,
    n_clusters=None,
    linkage='complete'
)

hc_comp.fit(X)
```

- `n_clusters=None`: no fijamos un número de clusters.
- `distance_threshold=0`: construimos todo el dendrograma.
- `linkage='complete'`: usamos complete linkage.

Average y single linkage

Podemos cambiar fácilmente el tipo de *linkage*.

Average linkage:

```
hc_avg = HClust(
    distance_threshold=0,
    n_clusters=None,
    linkage='average'
)
hc_avg.fit(X)
```

Single linkage:

```
hc_sing = HClust(
    distance_threshold=0,
    n_clusters=None,
    linkage='single'
)

hc_sing.fit(X)
```

Utilizando una matriz de distancias

También podemos proporcionar directamente una **matriz de distancias previamente calculada**.

Primero construimos la matriz de distancias Euclidianas:

```
D = np.zeros((X.shape[0], X.shape[0]))
```

```
for i in range(X.shape[0]):
    x_ = np.multiply.outer(
        np.ones(X.shape[0]), X[i]
    )
    D[i] = np.sqrt(
        np.sum((X - x_)**2, 1)
    )
```

En este ejemplo,

$$D \in \mathbb{R}^{50 \times 50}.$$

Construcción del dendrograma

Para graficar el dendrograma utilizamos:

```
from scipy.cluster.hierarchy import dendrogram
```

Sin embargo, `dendrogram()` necesita una representación conocida como **linkage matrix**.

AgglomerativeClustering no produce directamente esta matriz, por lo que se puede utilizar:

```
# !pip install ISLP
from ISLP.cluster import compute_linkage

linkage_comp = compute_linkage(hc_comp)
```

Graficar el dendrograma

Eliminamos inicialmente el coloreado automático de las ramas:

```
cargs = {
    'color_threshold': -np.inf,
    'above_threshold_color': 'black'
}
```

Después graficamos:

```
fig, ax = plt.subplots(1, 1, figsize=(8, 8))

dendrogram(
    linkage_comp,
    ax=ax,
    **cargs
)
```

Los números en la parte inferior del dendrograma identifican las observaciones.

Cortar visualmente el dendrograma

Podemos utilizar `color_threshold` para visualizar un corte del dendrograma.

Por ejemplo, para un corte a altura 4:

```
fig, ax = plt.subplots(1, 1, figsize=(8, 8))

dendrogram(
    linkage_comp,
    ax=ax,
    color_threshold=4,
    above_threshold_color='black'
);
```

Las ramas que se fusionan por encima de la altura 4 se muestran en negro.

Graficar los clusters en los datos

```
labels = cut_tree(
    linkage_comp,
    n_clusters=4
).reshape(-1)

plt.scatter(
    X[:, 0],
    X[:, 1],
    c=labels
)
```


Disimilitud basada en correlación

También podemos utilizar la **correlación** entre observaciones para definir una medida de disimilitud.

La correlación mide la similitud entre los valores de las características de dos observaciones.

¿Cuántas variables necesitamos?

Utilizar correlación entre observaciones sólo tiene sentido cuando tenemos al menos **tres características**.

Complete linkage con correlación

Finalmente construimos y graficamos el dendrograma:

```
linkage_cor = compute_linkage(hc_cor)

fig, ax = plt.subplots(1, 1, figsize=(8, 8))

dendrogram(linkage_cor, ax=ax, **cargs)

ax.set_title(
    "Complete Linkage with "
    "Correlation-Based Dissimilarity"
)
```

Graficar ejemplo 3D

```
labels = cut_tree(
    linkage_cor,
    n_clusters=4
).reshape(-1)

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(
    X[:, 0],
    X[:, 1],
    X[:, 2],
    c=labels
)

ax.set_xlabel('X1')
ax.set_ylabel('X2')
ax.set_zlabel('X3')
ax.set_title('Hierarchical Clustering with Correlation-Based
Dissimilarity')
```

¡Gracias!